

On the other hand, the MySQL cluster is:

- An in-memory (or main-memory) database system, which relies on the main memory for data storage, management and manipulation, to achieve better performance while querying the data.
- A shared-nothing architecture database, which stores data over multiple independent data nodes in a cluster, instead of shared data storage, with no single point of failure (SPOF).
- A distributed database system, which stores portions of the database on multiple nodes in the cluster, which is managed by the central distributed database management system. So even if a node goes down, data integrity is maintained. Moreover, scaling becomes a handy task with almost 99.99 per cent availability.
- Database updates are synchronously replicated between all the data nodes in the cluster, guaranteeing data availability in case of node failures.

MySQL Cluster Components

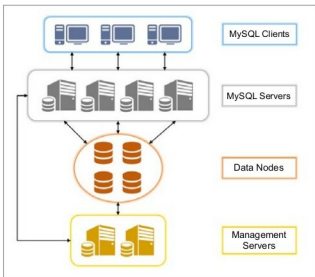


Figure 1: MySQL Cluster components

Management node/server

This maintains the cluster's global configuration file and provides cluster information whenever required. It also maintains logs of events happening in the cluster. The management client in the management node does all the administrative work, like starting/stopping nodes, starting/stopping backups and checking the cluster's status.

MySQL nodes/servers

These servers contain local configuration files. They run the *mysqld* daemon and group together to form a cluster, thus achieving high performance (due to parallelism) and high availability. These nodes cater to all incoming queries, communicate with data nodes and provide application access to the cluster.

Data nodes

These nodes run the *ndbd* daemon and are responsible for data storage and retrieval. Multiple data nodes come together to provide storage for the entire cluster, so that clients see them as a single database. Besides data storage, they keep monitoring other data nodes in the cluster and inform the management server in case of failures.

How it works

At the heart of the MySQL cluster, there lies the NDB (network database) storage engine, which is actually responsible for the high-available environment and data redundancy. In the basic scenario, we have an application that sends a query, usually as INSERT/UPDATE/DELETE-like SQL statements, to the MySQL server. In the MySQL cluster, one of the MySQL servers runs the NDB storage engine (or NDBCluster), which receives incoming SQL queries and communicates with data nodes to store the data. After confirming the successful writing of the data into the data nodes, the MySQL server acknowledges the application with an OK status.

In order to keep data available even after a node failure, it is divided into a number of chunks called partitions, which are equal to the number of nodes present in the cluster. So, each node has to store one partition along with a copy of a partition – this is called a replica. The number of the replica is mentioned in the configuration file on the management node. The MySQL cluster boasts of 99.999 per cent availability and replicas are key elements.

Handling failures

When a MySQL node fails, being a shared-nothing architecture, no other nodes (MySQL/data nodes or management nodes) in the cluster are affected — they continue doing their tasks. It's up to the application to connect to another MySQL node in the cluster. On the other hand, if a data node fails, another data node in the cluster takes over the responsibility and due to data redundancy (replicas), data will also be available. While the MySQL cluster takes care of node failures, you need to take care that the failed data node wakes up as early as possible, as you never know when other node(s) will stop working. Failure of the management node doesn't hamper the set-up much, as this node deals only with monitoring and backup tasks but then you might not be able to start/stop other cluster nodes. Having two management nodes is definitely a solution.

Implementation

Considering that I have three subnets and I do not have any budget issues, I opt to deploy four DNS-DHCP servers, of which three will be primary for their respective networks and the fourth will be secondary. I will have MySQL databases on all these four nodes (MySQL + Data nodes) and these are to be clustered such that they are in sync. For setting up the MySQL